In the following example, you call a computationally intensive method `def parsingDoc(doc: String): String`, which takes 1-3 seconds to execute. This method is a fake parsing; all it does is calculate the factorial of a relatively large number and output how long it took:

```
class CPUBoundedSpec ... {
  ...

  "calling to a local function sync" should {
    "eventually return" in {

      val result: Future[Done] =
        Source(1 to 10)
          .map(each => parsingDoc(each.toString))     ❶
          .run()

      Await.result(result, 30.seconds)
    }
  }
  ...
}
```

❶ Expensive operation

The source creates 10 elements and calls `parsingDoc(doc: String)` for each of them. It’s worth noting that you must wait for the result as an Akka stream runs in a separate thread from the test. If you don’t wait for the result, the test will finish before the stream finishes. Whether you wait or not, the stream does not stop until it has processed all the elements, you cancel it with a `Cancellable`, it throws an exception, or the JVM stops.

You can run this test and all the others shown later with the different `Specs` in the chapter16 project. Each section has a specific `Spec` with a similar name. The tests are marked `ignore`; to run a test,